

**Sistema de Resolución de Conflictos en una Negociación Automatizada: Modelado con
Grafos Dirigidos, Árboles de Decisión y Autómatas Finitos Deterministas**

Julio Aaron Andrade Chicaiza

Edwin David Guachi Cabera

Erick David Sabando Castillo

Anthony Alexander Vilaña Quishpe

Asesor

WASHINGTON EDUARDO LOZA HERRERA

Universidad de las Fuerzas Armadas ESPE

Departamento de Ciencias de la Computación

Programa

2026

Resumen

La teoría de juegos analiza la toma de decisiones estratégicas de un jugador cuando estas son dependientes de las decisiones de los demás jugadores, cada juego tiene sus particularidades. El informe presenta un sistema que modela una negociación automatizada entre varios participantes, quienes deben tomar decisiones sucesivas de cooperar, competir o ceder sabiendo que cada elección afecta las opciones siguientes de los demás. Se representó, evaluó y justificó los resultados de la negociación mediante recursos de matemática discreta.

Empezamos con un grafo dirigido, el cual se lo representa con su matriz de adyacencia, permite establecer quién influye sobre quién mediante su grado de entrada y salida, la forma en la que los elementos están conectados y la existencia de un ciclo hamiltoniano. Seguido de un árbol de decisión el cual enumera todos los casos posibles que puede haber de decisiones e identifica la estrategia conjunta que maximiza el puntaje total. Tercero, un autómata finito determinista, con los estados de *acuerdo*, *tensión* y *conflicto*, que resumió el avance de la negociación paso por paso y determina si el proceso llega hacia un acuerdo que ya no varía. El programa se implementó en lenguaje C++, empleando colas, recursividad y un generador aleatorio propio de cada decisión. Los resultados muestran que los tres modelos se complementan entre sí: el grafo explica que tan influenciados están unos con otros, el árbol proporciona la mejor decisión y el autómata recopila todo en una decisión.

Palabras clave: negociación, grafos dirigidos, árboles de decisión, autómatas finitos, teoría de juegos.

Abstract

Game theory analyzes a player's strategic decision-making when those decisions depend on the decisions of other players; each game has its own particular characteristics. This report presents a system that models an automated negotiation among several participants, who must make successive decisions to cooperate, compete, or yield, knowing that each choice affects the subsequent options of the others. The results of the negotiation were represented, evaluated, and justified using discrete mathematics.

We begin with a directed graph, represented by its adjacency matrix, which allows us to establish who influences whom through their in- and out-degrees, the way the elements are connected, and the existence of a Hamiltonian cycle. This is followed by a decision tree, which lists all possible decision cases and identifies the joint strategy that maximizes the total score. Third, a deterministic finite automaton, with states of agreement, tension, and conflict, summarizes the progress of the negotiation step by step and determines whether the process reaches a fixed agreement. The program was implemented in C++, using queues, recursion, and a random number generator for each decision. The results show that the three models complement each other: the graph explains how many elements influence each other, the tree provides the best decision, and the automaton summarizes everything into a single decision.

Keywords: negotiation, directed graphs, decision trees, finite automata, game theory.

Tabla de Contenido

Introducción	11
Justificación	12
Objetivos	13
Objetivo General.....	13
Objetivos Específicos	13
Marco Teórico y Diseño del Sistema.....	14
Fundamentos de Grafos, Árboles y Autómatas Finitos	14
Componentes del Sistema.....	14
Grafo Dirigido de Influencia entre Participantes	14
Árbol de Decisión y Búsqueda de la Mejor Estrategia	15
Autómata Finito Determinista del Estado de la Negociación	15
Figuras.....	16
Tablas	17
Conclusiones	18
Recomendaciones	19
Referencias Bibliográficas	20
Apéndices.....	21

Lista de Figuras

Figura 1 <i>Grafo Dirigido de Influencia entre Participantes</i>	16
---	----

Lista de Tablas

Tabla 1 <i>Tabla de Transición del Autómata Finito Determinista</i>	17
--	----

Lista de Apéndices

Apéndice A <i>Portada de la Presentación del Proyecto</i>	21
--	----

Introducción

Una negociación no suele resolverse en un solo paso, normalmente avanza por etapas sucesivas en las que cada persona decide siguiendo su propio interés y basando se en las decisiones que los demás tomaron antes. Modelar este tipo de proceso de forma computacional exige más que simular decisiones al azar; exige representar de manera explícita quién influye sobre quién, qué combinaciones de decisiones son posibles y cómo evoluciona el estado general de la negociación a medida que transcurren las etapas. Este documento presenta un sistema desarrollado en lenguaje C++ que aborda precisamente ese problema, combinando tres estructuras propias de la matemática discreta: grafos dirigidos, árboles de decisión y autómatas finitos deterministas.

Cada uno de estos tres formalismos cumple un papel distinto dentro del sistema. El grafo dirigido, representado mediante una matriz de adyacencia, describe la influencia de cada participante sobre el siguiente y permite analizar propiedades como el grado de entrada y salida, la conexidad y la existencia de un ciclo hamiltoniano. El árbol de decisión, construido de forma recursiva con memoria dinámica, enumera todas las combinaciones posibles de decisiones de los participantes y, mediante un recorrido exhaustivo de sus hojas, identifica cuál de esas combinaciones produce el mejor puntaje conjunto. Finalmente, el autómata finito determinista resume, etapa por etapa, si la negociación se está encaminando hacia un ACUERDO, si permanece en un estado de TENSION o si desemboca en CONFLICTO, aceptando la cadena de etapas solo cuando el proceso termina en el estado de acuerdo.

A lo largo de este documento se explican primero los fundamentos teóricos de grafos, árboles y autómatas finitos que sustentan el programa, y después se detalla, función por función, cómo se implementó cada uno de estos conceptos en `proyecto3.cpp`. Finalmente, se presentan las

conclusiones y recomendaciones derivadas de la simulación, con el objetivo de mostrar que un mismo problema de negociación puede analizarse desde tres perspectivas complementarias sin necesidad de recurrir a librerías externas de estructuras de datos.

Justificación

La elección de este tema responde a la necesidad de aplicar, en un solo proyecto integrador, tres bloques de contenidos que normalmente se estudian por separado: grafos, árboles y autómatas finitos. La resolución de conflictos en una negociación automatizada no es solo un ejercicio académico: procesos de decisión secuenciales con actores que se influyen entre sí aparecen en sistemas de negociación comercial, en la asignación automática de recursos y en la coordinación de agentes de software (Jennings et al., 2001), por lo que representarlos correctamente tiene consecuencias prácticas más allá del aula.

Además, gran parte del material didáctico habitual trata a los grafos, los árboles y los autómatas como temas aislados, cada uno con sus propios ejemplos desconectados entre sí. Este proyecto busca precisamente lo contrario: mostrar, con un solo caso de estudio (la negociación entre varios participantes), cómo estos tres formalismos se pueden combinar dentro de un mismo programa, donde la salida de un módulo (las decisiones tomadas) se convierte en la entrada del siguiente (el grafo de influencia, el árbol de juego y el autómata).

Finalmente, se justifica este trabajo por su valor formativo: exige implementar en C++ estructuras dinámicas propias (nodos con new/delete), recursividad, un recorrido en anchura manual sin la librería queue y una función de transición explícita para un autómata determinista, lo que fortalece tanto el manejo de memoria como el razonamiento algorítmico necesario para modelar problemas de decisión con múltiples actores.

Objetivos

Objetivo General

Desarrollar un programa en C++ con el fin de simular como es el proceso de una negociación entre varios actores que intervienen en dicha negociación, con el fin de evaluar los diferentes escenarios de las decisiones que se tomaron esto se lo realizara con tres estructuras matemáticas discretas las cuales son: Grafos dirigidos, Arboles de decisión y autómatas finitos, se utilizara estas estructuras para poder determinar con claridad si dicha negociación termina en un acuerdo estable en el cual se obtendrá una ganancia o si este escala hacia un conflicto causando perdidas.

Objetivos Específicos

Desarrollar un grafo dirigido para poder con este grafo representar y tener la idea de cómo influyen los participantes uno del otro, calculando así las múltiples conexiones de entrada y de salida de cada uno, comprobando que tan conectado está el grupo.

Construir un árbol de decisión para examinar todas las posibles combinaciones de elecciones (cooperar, competir o ceder) y se pueda identificar cuál de las diferentes estrategias es la que ofrece un mejor resultado para el grupo

Diseñar un autómata finito determinístico el cual clasifique el nivel de cooperación de los participantes en cada etapa y evalúe de forma eficiente y paso a paso si los participantes logran poner en estado de acuerdo final.

Marco Teórico y Diseño del Sistema

Fundamentos de Grafos, Árboles y Autómatas Finitos

Un grafo es una manera de representar relaciones entre elementos en los cuales los elementos son vértices y las relaciones entre ellos son aristas. Cuando cada arista tiene una dirección dada se habla de un grafo dirigido, que es útil para modelar una relación simétrica.

Un árbol, por otra parte, es un grafo sin ciclos, estructurado en niveles a partir de un nodo base; en programación suele construirse con recursividad.

Y, por último, un autómata finito es un modelo matemático que describe un procedimiento que puede estar en uno de varios estados y que cambia de estado al recibir símbolos como entrada, siguiendo una regla de transición que no varía; cuando esa regla asigna un único estado siguiente a cada par el autómata se denomina determinista (AFD), el cual se implementó en este proyecto. El programa construido combina tres modelos discretos que trabajan de coordinadamente sobre las mismas decisiones tomadas a lo largo de la negociación:

Componentes del Sistema

- Grafo dirigido: modela quién influye sobre quién a través de una matriz de adyacencia. Esta matriz mide grados de entrada y salida, la conexión entre nodos y ciclos del grafo.
- Árbol de decisión: modela todas las combinaciones de decisiones que los participantes pueden tomar. Mediante recorridos preorden y postorden, busca la mejor estrategia conjunta.
- Autómata finito determinista: modela el progreso del estado de la negociación, que pueden ser acuerdo, tensión o conflicto, paso a paso hasta llegar a una resolución de acuerdo o conflicto.

Grafo Dirigido de Influencia entre Participantes

En el sistema, cada participante de la negociación es un vértice y cada arista dirigida desde i hasta j significa que la decisión del participante i influye sobre la del participante j . El metodo `construirGrafoInfluencia()` representa la relación con una matriz de adyacencia: el primer paso es inicializar toda la matriz con ceros y luego, para cada participante i , crea la arista desde i hasta *siguiente*, donde $\text{siguiente} = (i + 1) \% \text{numParticipantes}$; usar el operador módulo hace que el último participante se conecte con el primero, cerrando un circuito.

Partiendo de esa matriz, los metodos `gradoSalida()` y `gradoEntrada()` cuentan, cada uno, cuántas aristas salen y cuántas llegan a cada vértice, mientras que `esConexo()` implementa un recorrido en anchura (BFS) construido con una estructura de tipo cola, para verificar, asumiendo el grafo como uno no dirigido, si se puede llegar desde cualquier vértice a todos los otros vértices. Y, para terminar, `tieneCicloCompleto()` sigue las aristas que salen del vértice 0 y comprueba si, tras recorrer `numParticipantes` pasos, el circuito regresa al vértice inicial sin repetir ningún vértice antes de tiempo, es decir un ciclo hamiltoniano.

Árbol de Decisión y Búsqueda de la Mejor Estrategia. La función `construirArbolDecision()` construye en la primera etapa el árbol de juego de manera recursiva: cada nivel en el árbol corresponde a la decisión de un participante específico y cada nodo tiene tres posibilidades: Cooperar, Competir, Ceder. Por ello con n participantes el árbol tiene 3^n hojas. Los nodos se crean de forma dinámica a llamar a `crearNodo()`. Cada nodo guarda la información del participante, con que opción se llegó al nodo y el puntaje que fue sumando desde la raíz. El caso base de este método recursivo ocurre cuando todos los participantes tomaron su decisión y en ese momento el nodo se vuelve hoja y regresa sin crear más hijos. Sobre este árbol se ejecutan los recorridos `recorrerPreorden()` es decir mostramos la raíz primero,

luego los hijos izquierdos y finalmente los hijos derechos, también ejecutamos el recorrido `recorrerPostorden()` es decir mostramos primero los hijos izquierdos, luego los hijos derechos y finalmente raíz.

Implementamos la función `buscarMejorHoja()` va por todas las hojas comparando sus puntos acumulados para guardar la mejor combinación de decisiones que dan los mejores resultados para el conjunto de negociadores, es decir, la mejor estrategia según la teoría de juegos, entendida como la solución que más beneficiara al conjunto de los negociadores (Nash, 1950); los parámetros `mejorPuntaje` y `mejorRuta` se pasan por referencia para preservar el resultado mientras ocurre el proceso de recursión. Al terminar, `liberarArbol()` recorre el árbol en postorden para liberar memoria con *delete*.

Autómata Finito Determinista del Estado de la Negociación. Se construyó el autómata con los estados de acuerdo, tensión y conflicto, un alfabeto con los símbolos alta, media y baja cooperación, un estado inicial TENSION y el estado de aceptación ACUERDO. En cada fase, `clasificarSimbolo()` cuenta los participantes que eligieron Cooperar y transforma el conteo en un símbolo del alfabeto, en caso de cooperar entre todos se asignó el símbolo alta cooperación, en el caso de que al menos la mitad colaboro se asignó el símbolo de media cooperación y en cualquier otro caso se asignó el símbolo de baja cooperación.

La función `transicion()` implementa de forma explícita la tabla de transición del AFD: para cada par que consta del estado actual y el símbolo leído existe un único estado siguiente, eso convierte al autómata en determinista. A grandes rasgos, la alta cooperación mantiene o mejora el estado en ACUERDO. Por otra parte, una cooperación baja solo aumenta CONFLICTO en cambio una cooperación media mantiene el sistema en TENSION. Finalmente, `simularAutomata()` interpreta todas las fases de la negociación como una secuencia de símbolos.

si el autómata llega al resultado final de ACUERDO, la secuencia de símbolos es aceptada y la negociación llega a un acuerdo sólido; si esto no se llega a dar el proceso no logró estabilizarse.

Un autómata finito determinista se define formalmente como una quintupla $(Q, \Sigma, \delta, q_0, F)$, donde Q es un conjunto finito de estados, Σ un alfabeto de entrada, δ la función de transición que asigna un único estado siguiente a cada par (estado, símbolo), q_0 el estado inicial y F el conjunto de estados de aceptación (Rabin y Scott, 1959).

La definición formal dada se observa en el código. Los estados y símbolos se implementan como constantes enteras. La función de transición δ corresponde a `transicion()`. El estado inicial q_0 se asigna a `TENSION` y el conjunto de estados finales F almacena el estado `ACUERDO`. Así, cada simulación completa de la negociación es igual a procesar una secuencia de símbolos del alfabeto, decidiendo así si la secuencia es aceptada o rechazada por el autómata.

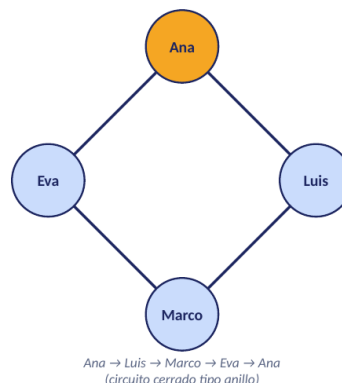
Figuras

Figura 1

Grafo Dirigido de Influencia entre Participantes

Grafo dirigido de influencia

- Cada participante es un vértice.
- Cada arista dirigida $i \rightarrow j$ significa "la decisión de i influye sobre j ".
- Se representa con una matriz de adyacencia: $\text{grafo}[i][j] = 1$ si existe la arista.
- Con el grado de entrada/salida medimos cuánto influye y es influido cada participante.



Nota. La figura muestra el grafo dirigido de influencia del sistema: cada participante es un vértice y las aristas $\text{Ana} \rightarrow \text{Luis} \rightarrow \text{Marco} \rightarrow \text{Eva} \rightarrow \text{Ana}$ forman un circuito cerrado tipo anillo, construido con la matriz de adyacencia. Elaboración propia a partir de la presentación del proyecto.

Tablas

Tabla 1

Tabla de Transición del Autómata Finito Determinista

Estado actual	alta_cooperacion	media_cooperacion	baja_cooperacion
ACUERDO	ACUERDO	TENSION	CONFLICTO
TENSION	ACUERDO	TENSION	CONFLICTO
CONFLICTO	TENSION	CONFLICTO	CONFLICTO

Nota. La tabla muestra la función de transición implementada en `transicion()`: para cada par (estado actual, símbolo leído) existe un único estado siguiente, lo que confirma que el autómata es determinista. Elaboración propia.

Conclusiones

Se concluyó que, al usar grafos, árboles y autómatas determinísticos, el proyecto logro analizar de forma exitosa la negociación desde varios ángulos. El grafo aclara la estructura de poder e influencia entre los participantes, el árbol de decisiones explora las múltiples rutas para luego elegir las más óptima (mejor jugada), y el autómata finito da el veredicto de la negociación y ayudara a probar si funciona o no

Se demostró que es posible desarrollar un sistema completo sin la necesidad de librerías externas o completas. Desarrollar las colas de búsqueda, manejo de memoria para los árboles y los generadores de números aleatorios permitió el control total de los recursos.

En la simulación se observó un efecto muy claro de contagio cuando un participante deside competir aumenta de manera exponencial la posibilidad de que los demás participantes también participen y quieran ganar por lo que esto puede llegar a un punto de conflicto, aunque las intenciones iniciales eran de cooperar.

Recomendaciones

Actualmente el árbol de decisiones solo evalúa la primera ronda o primer nivel para futuros trabajos se trata de ampliar su capacidad para que pueda analizar múltiples etapas de secuencia, integran secuencias de PODA para que el sistema obtenga más opciones pero que no se sobrecarga el sistema

También se recomienda mantener una estructura fija donde todos influyen por igual, se recomienda también permitir redes personalizadas para cada participante para poder obtener líderes de opinión dentro del grupo

Se sugiere agregar más estados distinguiendo así las posturas de ceder frente a competir realizando múltiples pruebas con diferentes configuraciones para cada participante de manera aleatoria para poder ver que tan probable es llegar a un acuerdo en distintas ocasiones.

Referencias Bibliográficas

Jennings, N. R., Faratin, P., Lomuscio, A. R., Parsons, S., Wooldridge, M. J., & Sierra, C.

(2001). Automated negotiation: Prospects, methods and challenges. *Group Decision and Negotiation*, 10(2), 199–215. <https://doi.org/10.1023/A:1008746126376>

Nash, J. F. (1950). The bargaining problem. *Econometrica*, 18(2), 155–162.

<https://doi.org/10.2307/1907266>

Rabin, M. O., & Scott, D. (1959). Finite automata and their decision problems. *IBM Journal of Research and Development*, 3(2), 114–125. <https://doi.org/10.1147/rd.32.0114>

Apéndices

Apéndice A

Portada de la Presentación del Proyecto



Nota. El apéndice muestra la portada de la presentación

Proyecto3_Negociacion_Automatizada.pptx, que resume los tres modelos aplicados: grafos dirigidos, árboles de decisión y autómatas finitos deterministas. Elaboración propia.